Chapter Six

Dynamic programming and Greedy Technique

Outline of the chapter

- General Method
  - Computing Binomial Coefficients (Reading Assignment)
  - Dynamic Programming
  - Greedy Technique
- Warshall's and Floyed's Algorithm
- Optimal Binary Search Tree
- Knapsack problem
- Prims Algorithm
- Kruskal's Algorithm

Dynamic Programming - General Method

- An algorithm design technique
- Dynamic Programming is also used in optimization problems. Like divide-and-conquer method,
- Dynamic Programming solves problems by combining the solutions of sub-problems.
- Moreover, Dynamic Programming algorithm solves each sub-problem just once and then saves its answer in a table, thereby avoiding the work of re-computing the answer every time.
- Two main properties of a problem suggest that the given problem can be solved using Dynamic Programming.
- These properties are **overlapping sub-problems** and **optimal substructure**.

Dynamic Programming - Overlapping Sub-Problems

- Similar to Divide-and-Conquer approach, Dynamic Programming also combines solutions to sub-problems.
- It is mainly used where the solution of one sub-problem is needed repeatedly.
- The computed solutions are stored in a table, so that these don't have to be re-computed.
- Hence, this technique is needed where overlapping sub-problem exists.
- For example,
  - Binary Search does not have overlapping sub-problem.
  - Whereas recursive program of Fibonacci numbers have many overlapping sub-problems.

Dynamic Programming - Optimal Sub-Structure

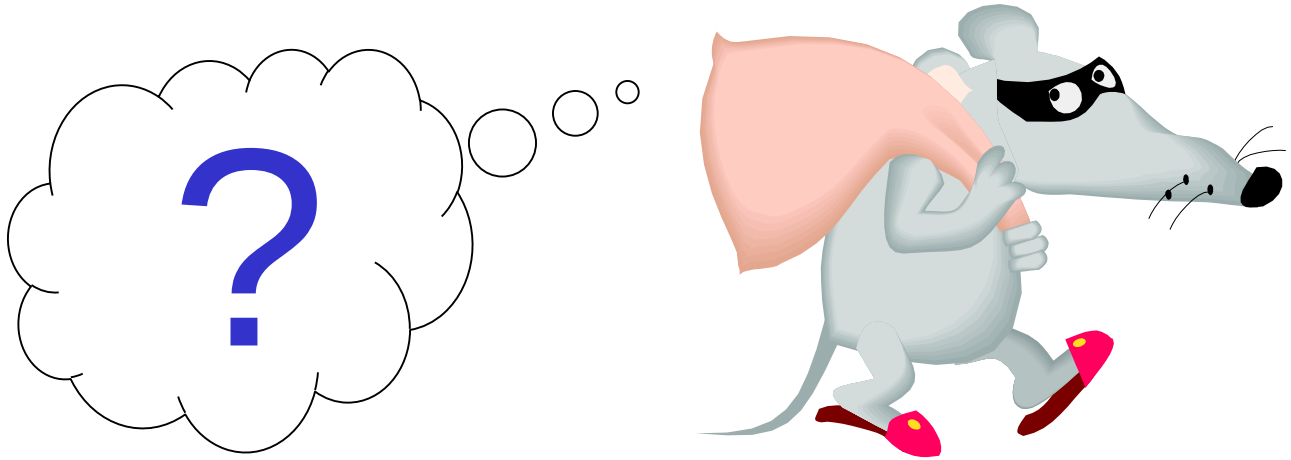
- A given problem has Optimal Substructure Property, if the optimal solution of the given problem can be obtained using optimal solutions of its sub-problems.
- For example,
  - The Shortest Path problem has the following optimal substructure property: If a node **x** lies in the shortest path from a source node **u** to destination node **v,** then the shortest path from **u** to **v** is the combination of the shortest path from **u** to **x,** and the shortest path from **x** to **v**.
- The standard All Pair Shortest Path algorithms like Floyd Warshall and Bellman-Ford are typical examples of Dynamic Programming.

Dynamic Programming - General Method

- Applicable when subproblems are **dependent**
  - Subproblems share subsubproblems
- Solves every subproblem ***just once*** and stores the answer in a table
- Used for **optimization problems**
  - A set of choices must be made to get an optimal solution
  - Find a solution with the optimal value (minimum or maximum)
  - There may be many solutions that lead to an optimal value
  - Our goal: **find an optimal solution**
- "Programming" here means "planning"

Knapsack Problem

- There are n different items in a store
- Item x_i :
  - weighs w_i pounds
  - worth \$ p_i
- A thief breaks in
- Can carry up to W pounds in his knapsack
- What should he take to maximize the value of his haul?



Knapsack Problem

- Each object x_i placed into the knapsack(capacity m) subject to the following constraints to obtain feasible solution.
  - maximize $\sum_{i=1}^{n} p_i x_i$ (maximize the profit)
  - Subject to $\sum_{i=1}^{n} w_i x_i \leq \mathrm{w}$ (capacity can't exceed w)
- The objective of the knapsack problem is try to maximize the profit by placing the objects into the knapsack.

Knapsack Problem Types

- 0-1 Knapsack Problem:
  - the items cannot be divided
  - thief must take entire item or leave it behind

  (0 – item not added; 1- item added to the knapsack)

- Fractional Knapsack Problem: *(Will cover on Greedy Technique Part)*
  - thief can take partial items
  - for instance, items are liquids or powders
  - solvable with a greedy algorithm…

0-1 Knapsack problem

- Given a knapsack with maximum capacity W, and a set S consisting of n items
- Each item i has some weight w_i and profit value p_i (all w_i and W are integer values)
- <u>Problem</u>: **How to pack the knapsack to achieve maximum total value of packed items**?
- Problem, in other words, is to find

$$\max \sum_{i \in T} p_i \text{ subject to } \sum_{i \in T} w_i \leq W$$

- The problem is called a "*0-1*" problem, because each item must be entirely accepted or rejected.

Dynamic Solution

- The dynamic solution for 0/1 knapsack problem is

$$F_n(M) = \max\left[\underset{\substack{\downarrow \\ \text{when } x_n = 1}}{F_{n-1}(M)}, \underset{\substack{\downarrow \\ \text{when } x_n = 0}}{F_{n-1}(M - W_n)} + P_n\right]$$

- When $x_n = 1$, the size of the bag is reduced by W_n which is the weight of the nth item. As we are placing the nth item we should add the profit for the last item.
- Similarly, we find $F_{n-1}(M)$ and so on up to $F_1(M)$
- If we try to remove the ith item then the profit of the ith item reduced from the total profit and weight of the ith item is reduced form the total weight which belongs to the profit and weight of $(i-1)^{th}$ item.

$$S_1^i = \left\{(P_i, W_i)/(P - P_i, W - W_i) \in S^{i-1}\right\}$$

Contd.

- Initially,

$$S_0^0 = \{0,0\}$$
$$S_1^i = S^{i-1} + (P_i, W_i)$$
$$S^i = S^{i-1} + S_1^i$$

Addition $S_1^i = S^i + (P_{i+1}, W_{i+1})$

Merging/Purging Rule $S^{i+1} = S^i \cup S_1^i$

- If $\mathbf{S^{i+1}}$ contains two pairs $\mathbf{(P_j, W_j)}$ and $\mathbf{(P_k, W_k)}$ with property that $\mathbf{P_j \leq P_k}$ and $\mathbf{W_j \geq W_k}$, then the pair **(Pj, Wj)** can be *discarded* or *eliminated*.
- Discarding or purging rule is also called as *Dominance Rule*.

Example

Consider instance of the knapsack problem: n=3, M=6;

$(p_1, p_2, p_3) = (1, 2, 5)$ & $(w_1, w_2, w_3) = (2, 3, 4)$

Initially, $S_0^0 = \{0,0\}$

S_1^1 can be ontained by selecting profit and weight of item1 and adding it S^0

$$S_1^1 = S^0 + (P_1, W_1) = \{(0,0) + (1,2)\} = \{1, 2\}$$

S^1 can be obtained by merging the pairs of S^0 *and* S_1^1

$$S^1 = S^0 \cup S_1^1 = \{(0,0) \cup (1,2)\} = \{(0,0), (1,2)\}$$

No elimination for pair (1,2) & (2,3)

$$S_1^2 = S^1 + (P_2, W_2) = \{(0,0)(1,2)\} + \{(2,3)\} = \{(2,3)(3,5)\}$$

$$S^2 = S^1 \cup S_1^2 = \{(0,0)(1,2)\} \cup \{(2,3)(3,5)\} = \{(0,0)(1,2)(2,3)(3,5)\}$$

$$S_1^3 = S^2 + (P_3, W_3) = \{(0,0)(1,2)(2,3)(3,5)\} + \{(5,4)\} = \{(5,4)(6,6)(7,7)(8,9)\}$$

$$S^3 = S^2 \cup S_1^3 = \{(0,0)(1,2)(2,3)(3,5)\} \cup \{(5,4)(6,6)(7,7)(8,9)\}$$

Contd.

consider pairs (3,5) and (5,4)

$3 \le 5$ and $5 \ge 4$ true, so discard (3,5)

$S^3 = \{(0,0)(1,2)(2,3)(5,4)(6,6)(7,7)(8,9)\}$

The pairs (7,7) and (8,9) can also be eliminated

because the weights of items chosen exceeds the knapsack capacity.

$\therefore S^3 = \{(0,0)(1,2)(2,3)(5,4)(6,6)\}$

In order to find the optimal solution, we must apply the following condition for the above optimal sequence.

If $(P_i, W_i) \in S^n$ and $(P_i, W_i) \notin S^{n-1}$ is true then select $x_n = 1$ otherwise $x_n = 0$

consider pair (6,6)

$(6,6) \in S^3$ and $(6,6) \notin S^2$ true, $\therefore x_3 = 1$

After selecting the object x_3, its profit & weight must be substracted

from total profit & weight.

$(6,6) - (5,4) = (1,2)$

Contd.

consider pair (1,2)

$(1,2) \in S^2 \text{and} (1,2) \notin S^1$ false, $\quad \therefore x_2 = 0$

consider again pair (1,2)

$(1,2) \in S^1 \text{and} (1,2) \notin S^0$ true, $\quad \therefore x_1 = 1$

After selecting the object x_1, its profit & weight must be substracted from total profit & weight.

$(1,2) - (1,2) = (0,0)$

Maximum/Total profit is $\sum (P_i, W_i) = P_1X_1 + P_2X_2 + P3X_3$

$$= 1(1) + 2(0) + 5(1) = 1 + 5 = 6$$

$\therefore$ Optimal solution is $(X_{1,} X_2, X_3) = (1,0,1)$.

Maximum Profit is $= 6$

All Pairs Shortest Path Problem (Floyd's Algorithm)

- **Problem:** In a weighted (di)graph, find shortest path from i to j for every pair of vertices i.e. ***finding shortest paths between all the pairs of vertices*** in the graph, G=(V,E).
- The weights of a path is measured by adding the weights of its constituent edges.
- We construct a cost adjacency matrix, obtained with the following conditions
  1. Cost (i,i)= 0
  2. Cost(i,j) is the cost associated with an edge <i,j> if there is an edge from vertex i to j.
  3. Cost(i, j) = ∞ if there is no edge between vertex i & j.
- idea: construct solution through series of matrices $A^{(0)}, \ldots, A^{(n)}$ using increasing subsets of the vertices allowed as intermediate

On the *k*-th iteration, the algorithm determines shortest paths between every pair of vertices *i, j* that use only vertices among 1,…,*k* as intermediate

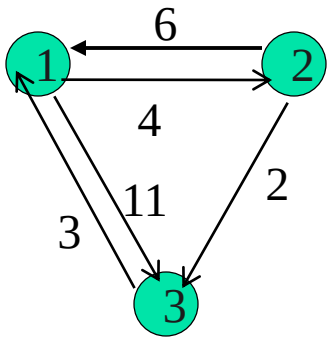
$$A^{k}[i,j] = \min_{1 \le k \le n} \left\{ A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j] \right\}$$

Floyd's Algorithm

```
Algorithm AllPaths(cost, A, n)
{
     for  i=1 to n do
       for j=1 to n do
          A[i,j] = cost[i,j]; // copy cost into A
     for k=1 to n do
       for i=1 to n do
          for j=1 to n do
             A^k[i,j] = min  A[i, j], { A[i,k]+A[k,j] }
}
```

Time complexity: $n^2+n^3=O(n^3)$

Example



Cost adjacency matrix

A^0	1	2	3
1	0	4	11
2	6	0	2
3	3	∞	0

Consider vertex 1 as intermediate vertex and find the shortest path for the remaining vertices 2 & 3

$$A^k[i,j] = \min_{1 \le k \le n} \{A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j]\}$$

$$A^1[2,3] = \min\{A^0[2,3], A^0[2,1] + A^0[1,3]\}$$

$$= \min\{2,\ 6+11\} = \min\{6,17\} = 6$$

$$i.e.\ 2->3 = 2$$

$$A^1[3,2] = \min\{A^0[3,2], A^0[3,1] + A^0[1,2]\}$$

$$= \min\{\alpha, 3+4\} = \min\{\alpha, 7\} = 7$$

$$i.e.\ 3->2 = 7$$

A^1	1	2	3
1	0	4	11
2	6	0	2
3	3	7	0

Contd.

Consider vertex 2 as intermediate vertex and find the shortest path for the remaining vertices 1 & 3

A^2	1	2	3
1	0	4	6
2	6	0	2
3	3	7	0

$$A^2[1,3] = \min\{A^1[1,3], A^1[1,2] + A^1[2,3]\}$$

$$= \min\{11,\ 4+2\} = \min\{11,6\} = 6$$

$$i.e. 1 -> 3 = 6$$

$$A^2[3,1] = \min\{A^1[3,1], A^1[3,2] + A^1[2,1]\}$$

$$= \min\{3,\ 7+6\} = \min\{3,13\} = 3$$

$$i.e. 3 -> 1 = 3$$

Contd.

Consider vertex 3 as intermediate vertex and find the shortest path for the remaining vertices 1 & 2

A^3	1	2	3
1	0	4	6
2	5	0	2
3	3	7	0

$$A^3[1,2] = \min\{A^2[1,2], A^2[1,3] + A^2[3,2]\}$$

$$= \min\{4,\ 6+7\} = \min\{4,13\} = 4$$

$$i.e. 1 -> 2 = 4$$

$$A^3[2,1] = \min\{A^2[2,1], A^2[2,3] + A^2[3,1]\}$$

$$= \min\{6,\ 2+3\} = \min\{6,5\} = 5$$

$$i.e. 2 -> 1 = 5$$

Example - Contd.

- In the above sequence, we must calculate the no of multiplications needed for M(i,k) and M(k+1, j)
- To compute matrices, the following recurrence formula is to be used.

$$M(i,j) = \begin{cases} 0 & \text{if } i = j \\ \min\left\{M(i,k) + M(k+1,j) + d_{i-1}d_k d_j\right\} & \text{if } i < j \end{cases}$$

Matrix-Chain Multiplication

Problem: given a sequence $\langle A_1, A_2, \ldots, A_n \rangle$, compute the product:

$$A_1 \cdot A_2 \cdots A_n$$

- In what order should we multiply the matrices?

$$A_1 \cdot A_2 \cdots A_n$$

- Parenthesize the product to get the order in which matrices are multiplied
- *E.g.:* $A_1 \cdot A_2 \cdot A_3 = ((A_1 \cdot A_2) \cdot A_3)$ **or**

  $= (A_1 \cdot (A_2 \cdot A_3))$
- Which one of these orderings should we choose?
  - The order in which we multiply the matrices has a significant impact on the cost of evaluating the product

Example

- Find the product of 4 matrices **A, B, C & D** with dimensions

  A = 13 x 5, B = 5 x 89, C = 89 x 3, D = 3 x 34

- Arrange the matrices in different combinations to find the no of multiplications needed for each combination.

1. Consider combination **M = ((AB)C)D**

   (AB) = 13x89x5 = 5785

   (AB)C = 13x3x89 = 3471

   ((AB)C)D = 13x34x3 = 1326

   Total = 10,582 multiplications are required

2. Consider another combination **M = (AB)(CD)**

   (AB) = 13x89x5 = 5785

   (CD) = 89x34x3 = 9078

   (AB) (CD) = 13x34x89 = 39338

   Total = 54,201 scalar multiplications

Example - Contd.

3. Consider another combination **M = (A(BC))D**

(BC) = 5x3x89 = 1335

A(BC) = 13x3x5 = 195

A(BC)D = 13x34x3 = 1326

Total = 2,856 scalar multiplications

- Among the above combinations, the **3rd combination is optimal solution**, because it takes less no of multiplications.
- In traditional approach, it is a difficult task to find all possible combinations and time consuming.
- Therefore, we could apply a dynamic programming (DP) technique to solve this problem.

Greedy Method

- Greedy design technique is primarily used in *Optimization problems*.
  - Optimization problems are problems where in we would like to find the best of all possible solutions.
  - In other words we need to find the solution which has the optimal (maximum or minimum) value satisfying the given constraints.

- The Greedy approach helps in constructing a solution for a problem through a *sequence of steps*
  - each step is considered to be a partial solution.
  - the partial solution is extended progressively to get the complete solution

Greedy Method

- Constructs a solution to an *optimization problem* step by step through a sequence of choices that are:

  Defined by an objective function and a set of constraints

  - ***feasible***, *i.e. satisfying the constraints*
  - ***locally optimal*** *(with respect to some neighborhood definition)*
  - ***irrevocable*** *i.e. not altered*

- For some problems, it yields a globally optimal solution for every instance. For most, does not but can be useful for fast approximations. We are mostly interested in the former case.

Applications of the Greedy Strategy

- Optimal solutions:
  - change making for “normal” coin denominations
  - minimum spanning tree (MST)
  - single-source shortest paths
  - simple scheduling problems
  - Huffman codes

- Approximations/heuristics:
  - traveling salesman problem (TSP)
  - knapsack problem
  - other combinatorial optimization problems

Change-Making Problem

Given unlimited amounts of coins of denominations $d_1 > \ldots > d_m$,
give change for amount n with the least number of coins

Q: What are the objective function and constraints?

Example: $d_1 = 25c$, $d_2 = 10c$, $d_3 = 5c$, $d_4 = 1c$ and $n = 48c$

Greedy solution: **<1, 2, 0, 3>**

Greedy solution is

- optimal for any amount and "normal'' set of denominations

  Ex: Prove the greedy algorithm is optimal for the above denominations.

- may not be optimal for arbitrary coin denominations

  For example, $d1 = 25c$, $d2 = 10c$, $d3 = 1c$, and $n = 30c$

Fractional Knapsack Problem - Example

Consider the following instance of the knapsack problem: n=3, m=20;

$(p_1, p_2, p_3) = (25,24,15)$ & $(w_1, w_2, w_3) = (18,15,10)$

Method 1: Arrange the items in increasing order of their weights i.e. fill the knapsack by an object with minimum weight first.

Item	Profit	Weight	Remaining capacity	Object Selected
x3	15	10	20-10=10	x3=1
x2	24	15	10-10=0	x2=2/3
x1	25	18	0	x1=0

Rem. Capacity/weight of x2 = 10/15 = 2/3

Feasible Solution is {x1, x2, x3} ={ 0, 2/3, 1}

Total Profit = $\sum_{i=1}^{n} p_i x_i = p_1x_1 + p_2x_2 + p_3x_3$

$$= 25(0) + 24(\frac{2}{3}) + 15(1)$$

$$= 16 + 15$$

$$= 31$$

Example – Contd.

Method 2: Arrange the profits of all items in decreasing order

Item	Profit	Weight	Remaining capacity	Object Selected
x1	25	18	20-18= 2	x1=1
x2	24	15	2-2 =0	x2 = 2/15
x3	15	10	0	x3=0

Feasible Solution is { 1,2/15,0}

Total Profit = $\sum_{i=1}^{n} p_i x_i = p_1 x_1 + p_2 x_2 + p_3 x_3$

$$= 25(1) + 24(\frac{2}{15}) + 15(0)$$

$$= 25 + \frac{48}{15}$$

$$= 28.2$$

Example – Contd.

Method 3: Arrange items in decreasing order of their profit-weight ratio

Item	P_i/w_i	Profit	Weight	Remaining capacity	Object Selected
x2	1.6	24	15	20-15= 5	x2=1
x3	1.5	15	10	5-5 =0	x3 = 1/2
x1	1.3	25	18	0	x1=0

x1= P1/w1 = 25/18=1.3
x2 = p2/w2 =24/15=1.6
x3= p3/w3= 15/10= 1.5

Feasible Solution is { 0, 1, 1/2}

Total Profit = $\sum_{i=1}^{n} p_i x_i = p_1 x_1 + p_2 x_2 + p_3 x_3$

$$= 25(0) + 24(1) + 15(\frac{1}{2})$$

$$= 24 + 7.5$$

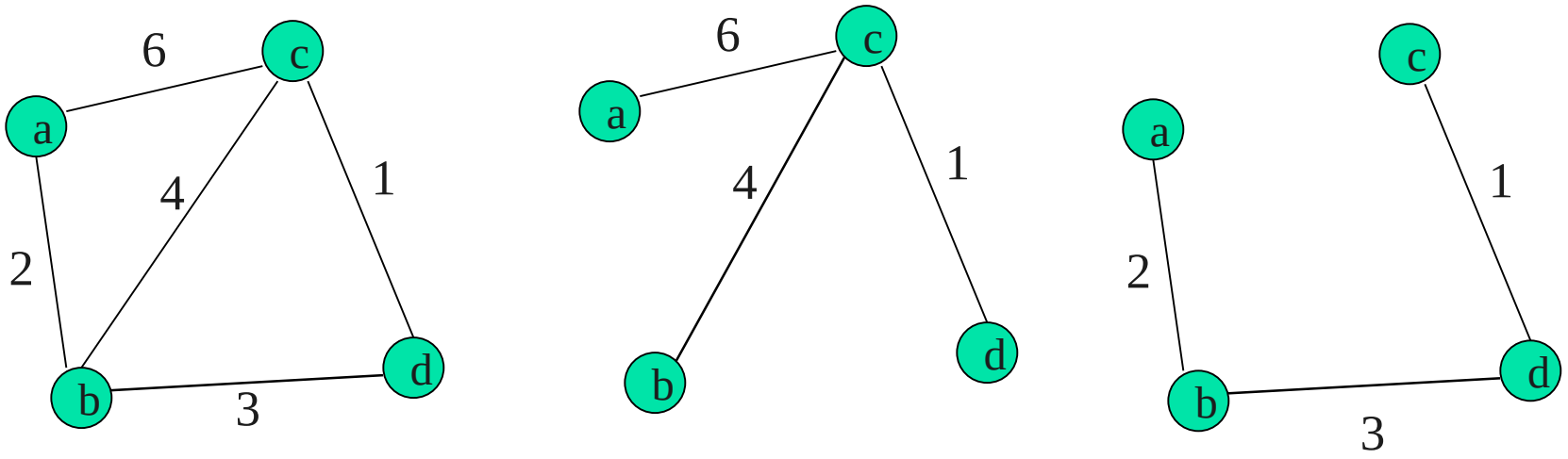
$$= 31.5$$

Minimum Spanning Tree (MST)

- *Spanning tree* of a connected graph *G*: a connected acyclic subgraph of *G* that includes all of *G*'s vertices

- *Minimum spanning tree* of a weighted, connected graph *G*: a spanning tree of *G* of the minimum total weight

- Example:



A Graph G Possible spanning tree minimum spanning tree

Contd.

Applications

- Useful in constructing networks
- Used to find airline routes
- Used to approximately solve travelling salesman problem

Approaches:

- Prim’s Algorithm
- Kruskal’s Algorithms

Prim's Algorithm for Finding an MST

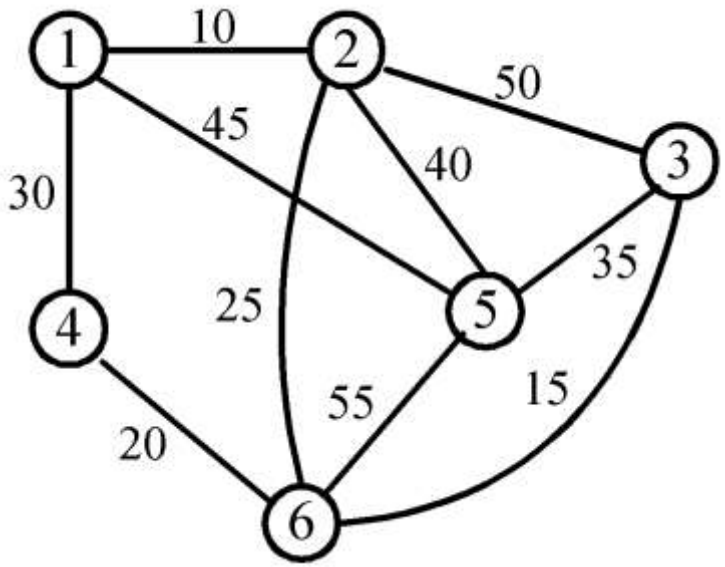
Step 1: $x \in V$, Let $A = \{x\}$, $B = V - \{x\}$.

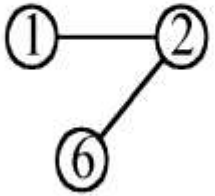
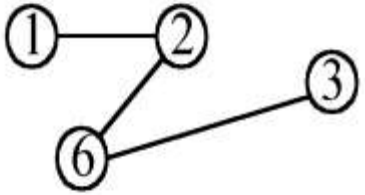
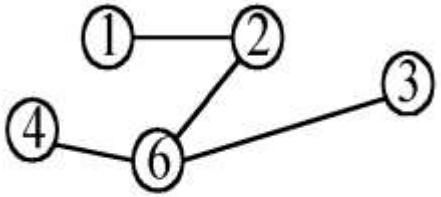
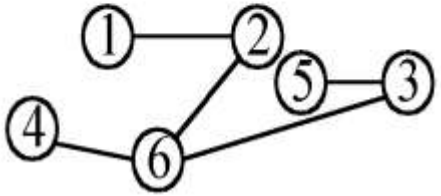
Step 2: Select $(u, v) \in E$, $u \in A$, $v \in B$ such that (u, v) has the smallest weight between A and B.

Step 3: Put (u, v) into the tree. $A = A \cup \{v\}$, $B = B - \{v\}$

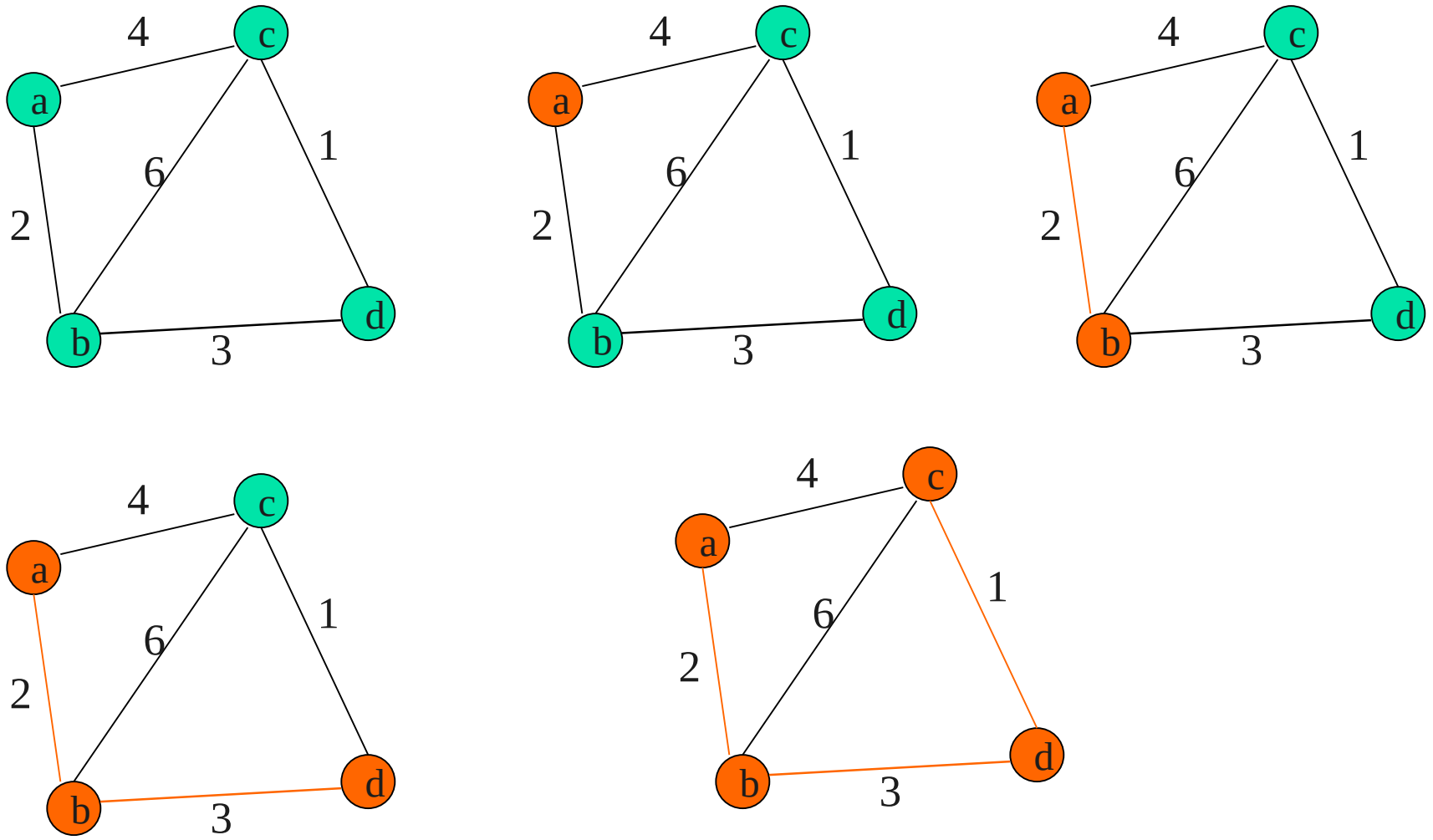
Step 4: If $B = \varnothing$, stop; otherwise, go to Step 2.

An Example for Prim's Algorithm



Edge	Cost	Spanning tree
(1,2)	10	
(2,6)	25	
(3,6)	15	
(6,4)	20	
(3,5)	35	

Example2



Prim's algorithm - Performance

- Needs priority queue for locating closest fringe vertex.

- Efficiency
  - **O(n^2)** for weight matrix representation of graph and array implementation of priority queue
  - **O(m log n)** for adjacency lists representation of graph with n vertices and m edges and min-heap implementation of the priority queue

Kruskal's MST Algorithm

- Sort the edges in nondecreasing order of lengths
- "Grow" tree one edge at a time to produce MST through a series of expanding forests $F_1, F_2, \ldots, F_{n-1}$
- On each iteration, add the next edge on the sorted list unless this would create a cycle. (If it would, skip the edge.)

Kruskal's Algorithm for Finding MST

Input : A weighted, connected and undirected graph

$G = (V, E)$.

Output : A minimal spanning tree for G.

$T := \phi$

While T contains less than n - 1 edges do

Begin

Choose an edge (v, w) from E of the smallest weight

Delete (v, w) from E

If (the adding of (v, w) to T does not create a cycle in T) then

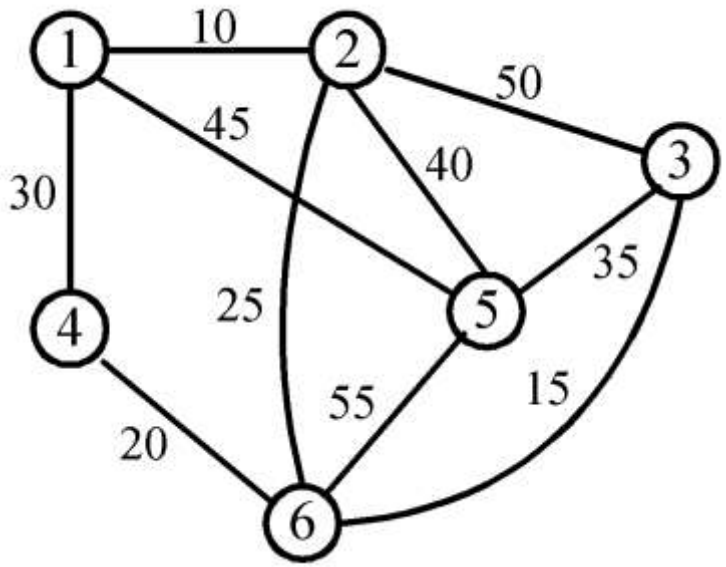
Add (v, w) to T

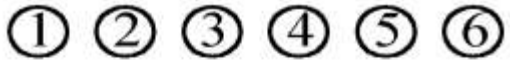
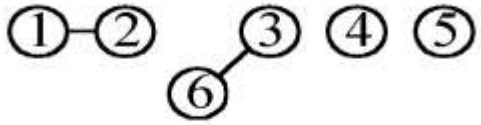
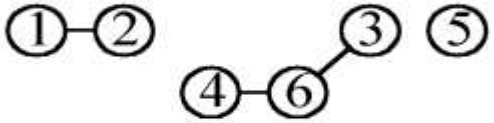
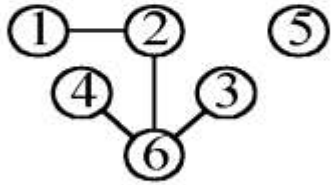
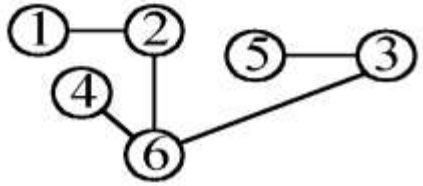
Else

Discard (v, w)

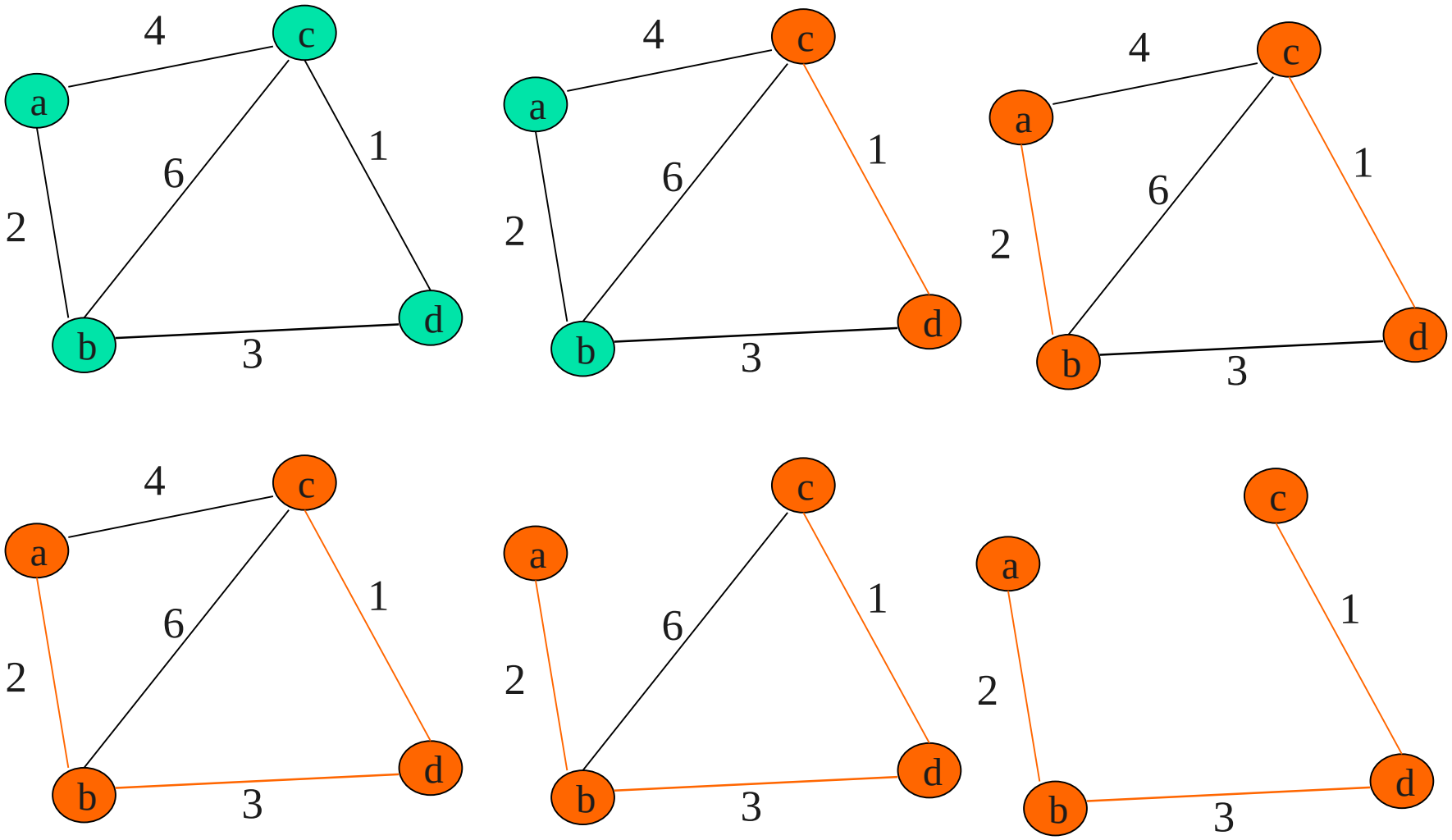
End

Example 1



Edge	Cost	Spanning Forest
		
(1,2)	10	
(3,6)	15	
(4,6)	20	
(2,6)	25	
(1,4)	30	(reject)
(3,5)	35	

Example2


4
c
a
6
1
2
b
3
d
4
c
a
6
1
2
b
3
d
4
c
a
6
1
2
b
3
d
4
c
a
6
1
2
b
3
d
c
a
6
1
2
b
3
d
c
a
1
2
b
3
d

Kruskal's algorithm - Performance

- Algorithm looks easier than Prim's but is harder to implement (checking for cycles!)
- Cycle checking: a cycle is created iff added edge connects vertices in the same connected component
- Runs in ***O(m log m)*** time, with $m = |E|$. The time is mostly spent on sorting.

Single-Source Shortest Paths (SSSP)

***Single Source Shortest Paths Problem*:** Given a weighted connected (directed) graph G, find shortest paths from source vertex *s* to each of the other vertices

***Dijkstra's algorithm*:** Similar to Prim's MST algorithm, with a different way of computing numerical labels: Among vertices not already in the tree, it finds vertex *u* with the smallest sum

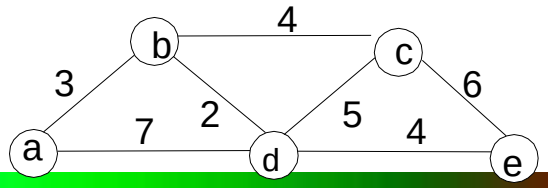
$$d_v + w(v,u)$$

where

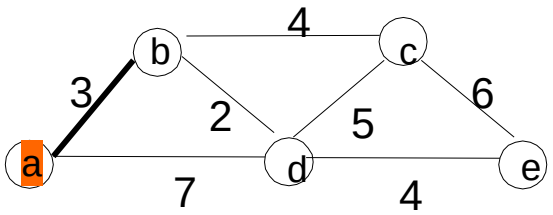
v is a vertex for which shortest path has been already found on preceding iterations (such vertices form a tree rooted at *s*) d_v is the length of the shortest path from source *s* to *v*

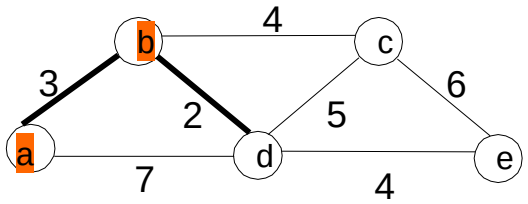
$w(v,u)$ is the length (weight) of edge from *v* to *u*

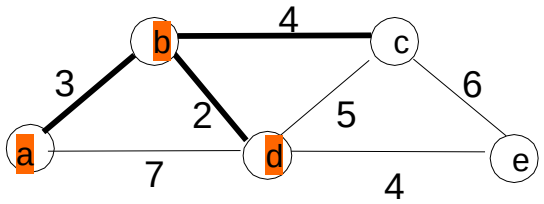
Example

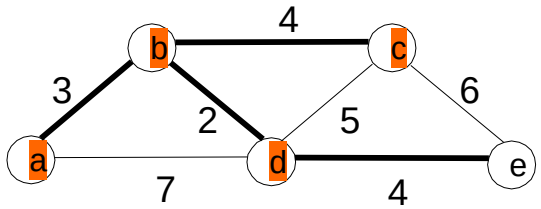


Tree vertices	Remaining vertices
a(-,0)	<u>b(a,3)</u> c(-,∞) d(a,7) e(-,∞)
b(a,3)	c(b,3+4) <u>d(b,3+2)</u> e(-,∞)
d(b,5)	<u>c(b,7)</u> e(d,5+4)
c(b,7)	<u>e(d,9)</u>
e(d,9)	



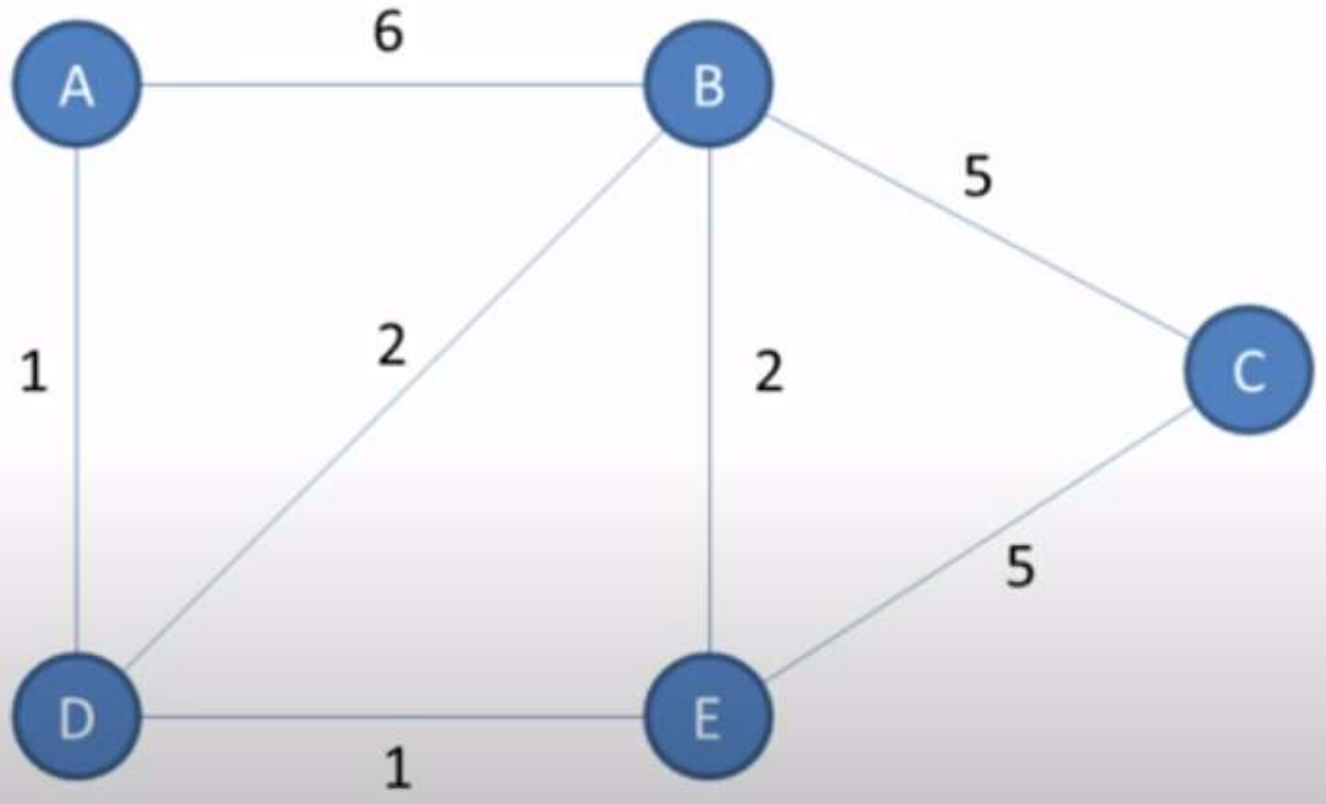






Example: Find the shortest distance using Dijkstra's Algorithm

Find the shortest path from vertex A to every other vertex



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Dijkstra's algorithm - Performance

- Applicable to both undirected and directed graphs

- Efficiency
  - $O(|V|^2)$ for graphs represented by weight matrix and array implementation of priority queue
  - $O(|E|\log|V|)$ for graphs represented by adj. lists and min-heap implementation of priority queue